

**PROPOSAL ALP
ADVANCED PROGRAMMING & DATA STRUCTURE**

***SISTEM INFORMASI ASPIRASI PUBLIK
PADA APLIKASI SIGAP***



Anggota Tim:

- | | |
|------------------------|---------------|
| 1. Arya Athaullah Rafi | 1076012510026 |
| 2. Cavin Amadeus | 1076012510021 |
| 3. Darren Wibisono | 1076012510025 |

**PROGRAM STUDI INFORMATIKA
SCHOOL OF INFORMATION TECHNOLOGY
UNIVERSITAS CIPUTRA
2025-2026**

DAFTAR ISI

DAFTAR ISI.....	1
BAB I	
PENDAHULUAN	3
1.1. Latar Belakang	3
1.2. Ruang Lingkup.....	4
1.2.1. Fitur.....	4
1.2.2. Batasan	5
1.2.3. Teknologi	5
BAB II	
LANDASAN TEORI	6
2.1 Linked List.....	6
2.2 Hash Map	7
2.3 Priority Queue.....	8
BAB III	
DESAIN SISTEM	10
3.1. Class Diagram.....	10
3.1.1 User (Abstract Class)	11
3.1.2 Citizen.....	11
3.1.3 Admin	11
3.1.4 Institution	11
3.1.5 InstitutionAdmin.....	11
3.1.6 Aspiration.....	11
3.1.7 CommentAspiration.....	11
3.1.8 MainFlow	12
3.1.9 Status (Enum).....	12
3.1.10 Priority (Enum)	12
3.2. Data Structure	13
3.3. Deskripsi Fitur Program.....	14
3.2.1 Fitur Register	14
3.2.2 Fitur Login	15
3.2.3 Fitur Logout	15
3.2.4 Fitur Tambah Aspirasi	16
3.2.4 Fitur Liat Aspirasi	17
3.2.5 Fitur Cari Aspirasi (Pergantian Struktur Data)	18
3.2.7 Fitur Upvote Aspirasi.....	18
3.2.8 Fitur Verifikasi Aspirasi (Penambahan data structure).....	19

3.2.9 Fitur Penentuan Prioritas.....	20
3.2.10 Fitur Distribusi Institusi (Tambahan data structure).....	23
3.2.11 Fitur Proses Laporan Institusi (Tambahan Data Structure)	24
3.2.12 Fitur Registrasi Admin Institusi.....	26
3.2.13 Fitur Dashboard Statistik	26
3.2.14 Fitur Komentar Aspirasi (Fitur Tambahan)	27
3.2.15 Fitur Lihat Antrian Dual Queue (Fitur Tambahan).....	28
3.2.16 Fitur Update Status Laporan (Fitur Tambahan).....	29
3.2.17 Fitur Riwayat Laporan Selesai (Fitur Tambahan)	29
3.2.18 Fitur Kelola Institusi (Fitur Tambahan).....	30
3.2.19 Fitur penyimpanan data (Fitur Tambahan)	31
BAB IV	
TIMELINE	32
DAFTAR PUSTAKA.....	34

BAB I

PENDAHULUAN

1.1. Latar Belakang

Partisipasi publik dalam pengawasan layanan pemerintah merupakan salah satu pilar penting dalam mewujudkan tata kelola yang transparan dan akuntabel [1]. Di Indonesia, SP4N-LAPOR! telah ditetapkan sebagai platform nasional satu pintu untuk menampung aspirasi dan pengaduan masyarakat terhadap layanan publik berdasarkan Peraturan Presiden Nomor 76 Tahun 2013 [2]. SP4N-LAPOR! dirancang sebagai sistem pengelolaan pengaduan publik yang terintegrasi secara nasional, dengan tujuan memberikan akses partisipasi masyarakat dalam menyampaikan pengaduan serta meningkatkan akuntabilitas dan responsivitas pemerintah dalam penyelenggaraan layanan publik [2].

Meski telah berjalan lebih dari satu dekade, sistem ini masih menghadapi hambatan fundamental. Berdasarkan data KemenPANRB tahun 2022, rata-rata waktu penyelesaian satu laporan membutuhkan sekitar enam hari, dengan hanya delapan analis yang menangani lebih dari 8.000 aspirasi setiap harinya secara manual [3]. Hingga tahun 2022, sebanyak 8,61% instansi pusat maupun daerah belum terhubung ke SP4N-LAPOR!, yang menunjukkan bahwa integrasi sistem pengaduan publik secara nasional belum sepenuhnya terwujud [4]. Data pengaduan yang masuk lebih banyak digunakan untuk pelaporan administratif dan pemantauan kinerja, bukan sebagai bukti analitis untuk mendukung pengambilan keputusan kebijakan, dengan hambatan utama berupa fragmentasi kelembagaan, keterbatasan interoperabilitas sistem teknis, dan kapasitas analitis yang tidak memadai [5].

Metode tradisional penanganan pengaduan yang mengandalkan proses manual, alur birokrasi yang panjang, serta ketiadaan pembaruan status secara berkala terbukti tidak memadai untuk kebutuhan modern, dan secara langsung merusak kepercayaan publik terhadap institusi pemerintah [6]. Kondisi ini terkonfirmasi secara empiris, di mana berdasarkan indeks kepercayaan warga yang dipublikasikan Tempo pada tahun 2026, tingkat kepercayaan masyarakat terhadap DPR hanya mencapai 56% dan terhadap DPRD sebesar 65% [7]. Partisipasi warga dalam e-government sangat penting bagi pemerintah karena mencerminkan keterlibatan aktif masyarakat

dalam proses kebijakan publik, dan rendahnya kepercayaan publik menjadi penghalang utama partisipasi tersebut [8].

Permasalahan ini selaras dengan tantangan yang diidentifikasi dalam *Sustainable Development Goals* (SDGs), khususnya SDG 16 yaitu *Peace, Justice and Strong Institutions*, yang mendorong terwujudnya institusi yang efektif, akuntabel, dan inklusif di semua tingkatan pemerintahan [9]. Inisiatif digital seperti SP4N-LAPOR! merupakan bagian dari upaya pemerintah untuk memperkuat akuntabilitas dan transparansi, di mana platform-platform ini tidak hanya menyederhanakan pemberian layanan tetapi juga memperkenalkan mekanisme umpan balik yang memungkinkan warga menyampaikan pengaduan, memantau permintaan, dan mengevaluasi layanan [10].

Berdasarkan seluruh permasalahan tersebut, sistem **SIGAP** (Sistem Informasi Aspirasi Publik) diusulkan sebagai solusi yang mampu menerima laporan dari masyarakat, memprioritaskannya berdasarkan skor, dan meneruskannya ke instansi pemerintah yang relevan tanpa bergantung pada proses manual.

1.2. Ruang Lingkup

Berikut merupakan fitur, batasan serta teknologi dalam ruang lingkup project ALP kami.

1.2.1. Fitur

Program SIGAP memiliki fitur-fitur berikut yang dapat diakses sesuai dengan peran masing-masing pengguna.

- a. **Citizen** dapat melakukan registrasi, login dan logout dari sistem, membuat laporan aspirasi baru dengan mengisi judul, deskripsi, kategori, dan lokasi, melihat seluruh laporan publik beserta status, memberikan upvote pada laporan milik pengguna lain (satu kali per laporan), serta memantau perkembangan status laporan yang telah dibuat.
- b. **Admin** dapat melakukan login dan logout dari sistem, melihat seluruh aspirasi yang masuk, memverifikasi atau menolak laporan (mengubah status menjadi APPROVED atau MENOLAK), menentukan tingkat prioritas laporan (1 sampai

10), menentukan institusi tujuan penerima laporan, menghapus laporan yang tidak sesuai, serta menambah akun admin lain. Admin juga dapat mengakses dashboard yang menampilkan total laporan, laporan selesai, laporan pending, kategori terbanyak, jumlah laporan per institusi, serta laporan dengan upvote tertinggi.

- c. Institusi** dapat melakukan login dan logout dari sistem, melihat antrean laporan yang telah diteruskan oleh admin, memproses laporan berdasarkan urutan prioritas tertinggi, memperbarui status laporan menjadi ON_PROGRESS atau DONE serta menambah akun admin institusi dengan kategori yang sama.

1.2.2. Batasan

Program ini tidak mencakup koneksi ke basis data eksternal maupun penggunaan framework UI dinamis seperti JavaFX atau JavaSwing. **Seluruh data tersimpan menggunakan .txt sehingga jika program dijalankan ulang, data masih tersimpan.**

1.2.3. Teknologi

Program SIGAP dikembangkan menggunakan bahasa pemrograman Java berbasis antarmuka baris perintah (CLI/Terminal), tanpa antarmuka grafis. Kami juga menggunakan beberapa data struktur untuk diimplementasikan pada beberapa fitur program seperti Hashmap, LinkedList dan Priority Queue.

-

BAB II

LANDASAN TEORI

Sistem pengaduan masyarakat merupakan platform yang digunakan untuk menerima, mengelola dan menindaklanjuti aspirasi dan laporan yang diajukan masyarakat terkait isu-isu yang terjadi. Sistem ini bertujuan untuk meningkatkan transparansi, partisipasi publik dan efektivitas pelayanan pemerintah dalam menangani masalah masyarakat. Oleh karena itu diperlukan penggunaan data structure data yang efisien agar performa sistem tetap optimal meskipun data dalam jumlah besar.

Struktur data adalah cara untuk mengorganisir dan menyimpan data di komputer sehingga data dapat diakses dan digunakan secara efisien [15]. Pemilihan struktur data yang tepat merupakan keputusan yang desain kritis yang secara langsung mempengaruhi kompleksitas waktu dan ruang dari keseluruhan sistem [15]. Pada sistem SIGAP digunakan tiga struktur data utama.

- LinkedList
- HashMap
- Priority Queue

2.1 Linked List

Linked list adalah sebuah data Structure linear yang terdiri dari rangkaian *node*, dimana setiap *node* menyimpan dua komponen: data yang berisi nilai yang disimpan, dan *pointer* yang berisi referensi pada lokasi *node* selanjutnya dalam rangkaian [14]. *node* terakhir dalam linked list (*tail*) memiliki pointer bernilai *null* sebagai penanda akhir rangkaian. Berbeda dengan array yang menggunakan blok memori kontinu, linked list melakukan alokasi memori secara lebih dinamis sehingga ukuran dapat bertumbuh dan menyusut saat program berjalan tanpa perlu deklarasi ukuran di awal [13].

Terdapat 3 varian utama linked list yaitu single Linked List dimana setiap *node* hanya memiliki 1 *pointer* menuju *node* selanjutnya untuk *transversal* hanya satu arah. Double linked List dimana setiap *node* memiliki 2 *pointer* yaitu *prev* dan *next* yang

memungkinkan *transversal* 2 arah. Circular Linked List dimana sebuah *linked list* dimana *node* terakhir menunjuk lagi ke *node* pertama membentuk sebuah lingkaran.[14]

Linked list memiliki kemampuan dalam efisiensi operasi penambahan dan penghapusan. Bila posisi target sudah diketahui, kedua operasi tersebut hanya memerlukan pembaruan pointer tanpa perlu menggeser elemen lain, sehingga berjalan dalam waktu konstan $O(1)$ [13]. Sebaliknya, karena tidak ada indeks langsung, operasi pencarian dan akses ke elemen tertentu memerlukan traversal sekuensial dari awal dengan kompleksitas $O(n)$ [14].

Kelebihan:

- Alokasi memori bersifat dinamis sehingga ukuran struktur data bisa disesuaikan saat program berjalan dan tidak ada pemborosan memori akibat kapasitas berlebih [14].
- Operasi insert dan delete di posisi yang diketahui $O(1)$, lebih efisien daripada array yang membutuhkan $O(n)$ untuk menggeser seluruh elemen [13].
- Linked List ideal untuk pencatatan riwayat perubahan.

Kekurangan:

- Tidak mendukung akses acak yang menyebabkan elemen ke -n hanya dapat diakses melalui transversal dari awal, sehingga kompleksitasnya $O(n)$.
- Setiap *node* menyimpan satu atau dua *pointer* tambahan, menambah *overhead* memori dibandingkan dengan array murni [13].

2.2 Hash Map

HashMap atau *hash table* adalah struktur data yang mengimplementasikan pemetaan asosiatif antara pasangan kunci dan nilai (*key-value pairs*). Setiap kunci ditransformasi menjadi indeks array, sehingga operasi pencarian, penyisipan, dan penghapusan dapat dilakukan dalam waktu rata-rata $O(1)$ [11]. Karakteristik akses $O(1)$ ini menjadikan HashMap secara rata-rata jauh lebih cepat dibandingkan dengan struktur data terurut seperti *binary search tree* yang memerlukan $O(\log n)$ untuk operasi yang sama [11]. Fungsi hash $h(k)$ memetakan kunci k ke indeks integer dalam rentang $[0, m-1]$, di mana m adalah kapasitas tabel [11] $h(k) = \text{hash_code}(k) \bmod m$. Kondisi collision terjadi ketika dua kunci berbeda menghasilkan indeks yang sama. Dua metode utama untuk menanganinya adalah separate Chaining setiap bucket menyimpan struktur sekunder (linked list atau dynamic array) untuk menampung elemen yang bertabrakan indeks. Open Addressing bila terjadi *collision*, algoritma mencari slot kosong berikutnya menggunakan *linear probing*, *quadratic probing*, atau *double hashing*. [11] Studi komparatif terhadap kedua pendekatan *separate chaining* menunjukkan bahwa

implementasi menggunakan *dynamic array* per bucket menghasilkan akses memori yang lebih konsisten dan konsumsi energi yang lebih rendah pada lingkungan *multithreaded* dibandingkan linked list, karena lokalitas memori yang lebih baik [7].

Load factor ($\alpha = n/m$) adalah rasio antara jumlah elemen dan kapasitas tabel. Ketika α melampaui ambang batas tertentu umumnya sekitar 0.75 pada banyak implementasi sistem melakukan *rehashing* dengan memperbesar kapasitas tabel untuk mempertahankan performa $O(1)$ [13]. *Worst case* $O(n)$ terjadi bila semua kunci terpetakan ke satu bucket akibat fungsi hash yang buruk [11].

Kelebihan:

- Operasi pencarian, penyisipan, dan penghapusan rata-rata $O(1)$, menjadikannya struktur data paling efisien untuk kebutuhan *lookup* berbasis kunci [11].
- Cocok untuk implementasi pemetaan deterministik kategori-ke-tujuan (routing) karena setiap pencarian kunci menghasilkan nilai dalam waktu konstan [15].

Kekurangan:

- Tidak mempertahankan urutan penyisipan iterasi atas HashMap menghasilkan urutan yang tidak dapat diprediksi sehingga tidak cocok untuk kebutuhan data terurut [14].
- Performa menurun drastis bila *load factor* tinggi atau fungsi hash buruk menyebabkan banyak *collision* sehingga *worst case* mencapai $O(n)$ [7].
- Membutuhkan alokasi memori untuk seluruh kapasitas bucket meski banyak slot kosong, menambah overhead ruang [13].

2.3 Priority Queue

Priority Queue adalah tipe data abstrak yang menyimpan sekumpulan elemen di mana setiap elemen memiliki nilai prioritas yang menentukan urutan pemrosesannya. Berbeda dengan antrian konvensional yang mengikuti prinsip *First-In First-Out* (FIFO), *priority queue* selalu mengeluarkan elemen dengan nilai prioritas tertinggi terlebih dahulu, tanpa memandang urutan kedatangan [11]. Implementasi *priority queue* yang paling umum dan efisien adalah *binary max-heap*, yaitu pohon biner lengkap (*complete binary tree*) yang memenuhi *heap property*: setiap node induk selalu memiliki nilai lebih besar atau sama dengan kedua node anaknya [11]. *Max-heap* direpresentasikan dalam array satu dimensi. Untuk setiap elemen pada indeks i , hubungan antar node diformulasikan sebagai berikut [11]:

- Parent dari node $i = \lfloor (i - 1) / 2 \rfloor$
- Left child dari node $i = 2i + 1$

- Right child dari node $i = 2i + 2$

Dua operasi yang menjaga heap property setelah setiap modifikasi adalah heapify-Up dijalankan setelah insert. Elemen baru ditempatkan di ujung array lalu secara berulang ditukar dengan induknya selama nilainya lebih besar. Heapify-Down dijalankan setelah extract-max. Akar digantikan oleh elemen terakhir, lalu elemen tersebut ditukar dengan anak yang lebih besar hingga posisinya tepat.[11]

Kelebihan:

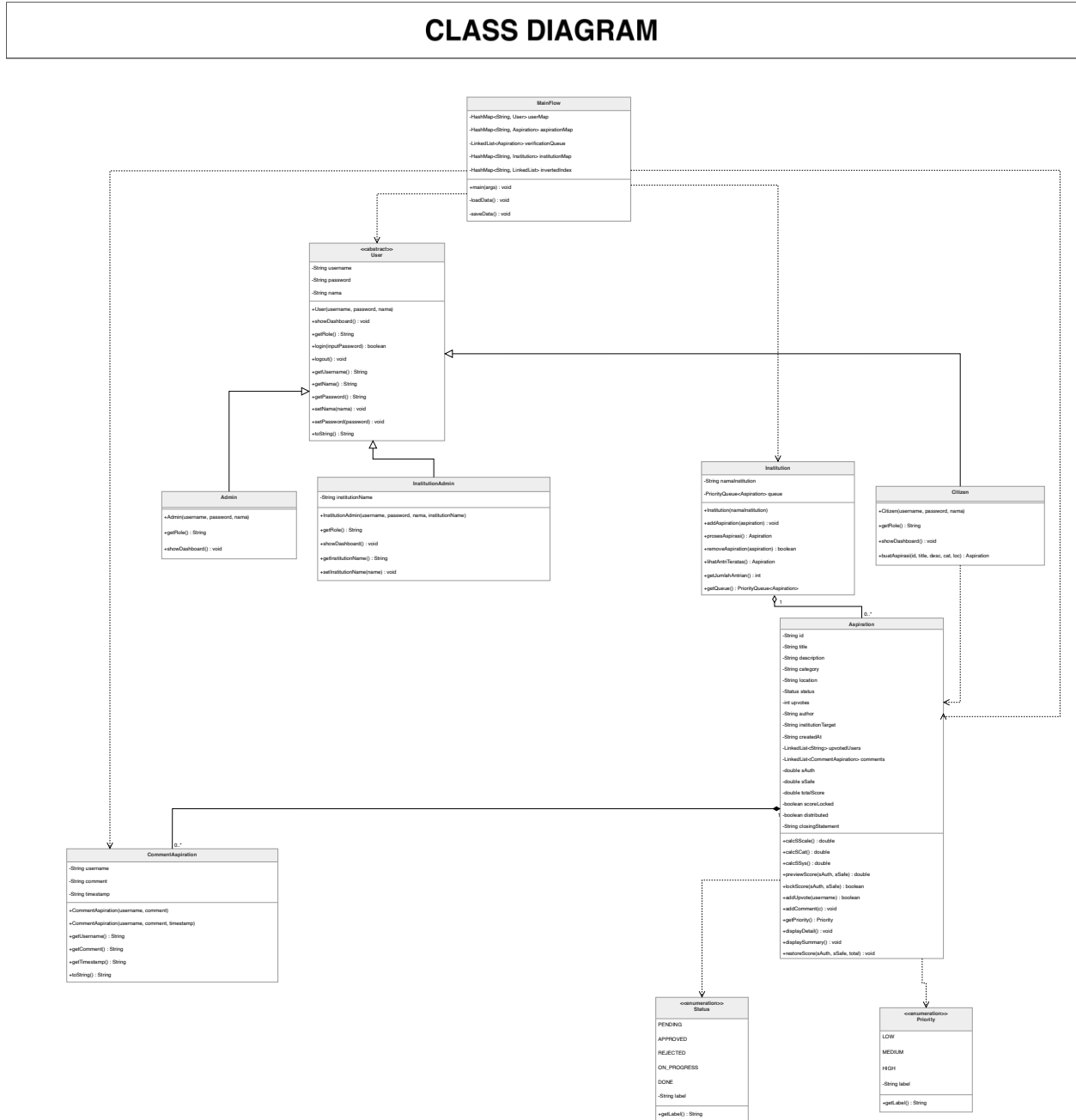
- Operasi insert dan extract-max berjalan $O(\log n)$, jauh lebih efisien daripada sorted array yang memerlukan $O(n)$ untuk setiap penyisipan baru [2].
- Operasi peek $O(1)$ memungkinkan sistem selalu mengetahui laporan berprioritas tertinggi tanpa biaya pemrosesan [11].

Kekurangan:

- Tidak mendukung pencarian elemen sembarang menemukan elemen non-prioritas memerlukan $O(n)$ [11].
- Hanya menjamin elemen teratas adalah yang tertinggi urutan elemen lainnya tidak terjamin sepenuhnya [15].

BAB III DESAIN SISTEM

3.1. Class Diagram



Gambar 3.1. Class Diagram

3.1.1 User (Abstract Class)

Class abstrak yang menjadi parent dari seluruh pengguna sistem. Class ini menyimpan atribut umum seperti username, password, dan nama.

3.1.2 Citizen

Class yang merepresentasikan masyarakat umum pengguna aplikasi. Citizen dapat membuat aspirasi, melihat laporan, dan memberikan upvote pada laporan lain.

3.1.3 Admin

Class yang bertugas mengelola seluruh laporan aspirasi dalam sistem. Admin dapat memverifikasi laporan, menentukan prioritas, dan mendistribusikan laporan ke institusi terkait.

3.1.4 Institution

Class yang merepresentasikan instansi pemerintah penerima laporan masyarakat. Institution menyimpan antrean laporan menggunakan PriorityQueue berdasarkan tingkat prioritas.

3.1.5 InstitutionAdmin

Class khusus admin pada suatu institusi tertentu. InstitutionAdmin bertugas memproses dan menyelesaikan laporan yang diterima institusinya.

3.1.6 Aspiration

Class utama yang merepresentasikan laporan atau aspirasi masyarakat. Class ini menyimpan informasi laporan seperti judul, deskripsi, status, prioritas, dan jumlah upvote.

3.1.7 CommentAspiration

Class yang digunakan untuk menyimpan komentar pada suatu laporan aspirasi. Komentar digunakan untuk meningkatkan transparansi dan diskusi publik pada laporan.

3.1.8 MainFlow

Class yang mengatur seluruh alur utama aplikasi berbasis CLI. MainFlow menangani menu, login, register, navigasi dashboard, serta input dan output terminal.

3.1.9 Status (Enum)

Enum yang digunakan untuk menyimpan status laporan pada sistem. Status terdiri dari PENDING, APPROVED, REJECTED, ON_PROGRESS, dan DONE.

3.1.10 Priority (Enum)

Enum yang digunakan untuk menentukan tingkat prioritas laporan. Priority terdiri dari HIGH, MEDIUM, dan LOW.

3.2. Data Structure

3.2.1. **LinkedList**

LinkedList dipilih karena mendukung operasi insert dan delete di kedua ujung dengan kompleksitas $O(1)$, sangat cocok untuk pola FIFO (First In, First Out) pada antrean verifikasi laporan dan penyimpanan daftar user yang telah melakukan upvote.

3.2.2. **HashMap**

Dalam SIGAP, HashMap digunakan dalam kasus: `HashMap<String, User>` untuk penyimpanan akun, `HashMap<String, Aspiration>` untuk penyimpanan laporan, `HashMap<String, Institution>` untuk pengelolaan institusi, `HashMap<String, LinkedList<String>>` untuk inverted index pencarian, serta beberapa HashMap tambahan sebagai cache statistik $O(1)$.

3.2.3. **PriorityQueue**

Dalam SIGAP, setiap Institution memiliki `PriorityQueue<Aspiration>` tersendiri. Laporan dengan totalScore tertinggi selalu berada di posisi teratas, memastikan institusi selalu memproses laporan dengan urgensi tertinggi lebih dulu.

3.2.4. **InvertedIndex (Data Structure Tambahan)**

Dalam SIGAP, inverted index diimplementasikan sebagai `HashMap<String, LinkedList<String>>` di mana key adalah kata kunci dan value adalah LinkedList berisi ID aspirasi yang mengandung kata tersebut. Setiap aspirasi diindeks saat dibuat. Fitur ini menggantikan pencarian berdasarkan ID langsung pada proposal awal, memberikan pengalaman pencarian yang jauh lebih natural dan fleksibel.

3.3. Deskripsi Fitur Program

3.2.1 Fitur Register

Fitur register digunakan ketika citizen ingin membuat akun baru pada sistem SIGAP agar dapat menggunakan seluruh fitur aplikasi.

Input

- Username
- Password
- Nama

Proses

1. Citizen memasukkan nama lengkap, username, dan password.
2. Sistem memvalidasi: nama tidak boleh kosong, username tidak mengandung spasi dan belum terdaftar, password minimal 6 karakter, konfirmasi password harus cocok.
3. Jika valid, sistem membuat object Citizen baru.
4. Data user disimpan ke dalam `HashMap<String, User>`.
5. Data langsung disimpan ke `users.txt`.

Output

- Akun berhasil dibuat.
- User dapat melakukan login ke sistem.

Struktur Data yang Digunakan

`HashMap<String, User>`

Alasan Penggunaan

`HashMap` digunakan agar proses pencarian username saat registrasi berjalan cepat dengan kompleksitas rata-rata $O(1)$.

3.2.2 Fitur Login

Fitur login digunakan ketika pengguna ingin masuk ke sistem sesuai role masing-masing.

Input

- Username
- Password

Proses

1. User memasukkan username dan password.
2. Sistem mencari username pada HashMap user.
3. Sistem mencocokkan password yang dimasukkan.
4. Jika sesuai, sistem menampilkan dashboard sesuai role pengguna.

Output

- User berhasil masuk ke dashboard sistem.

Struktur Data yang Digunakan

HashMap<String, User>

Alasan Penggunaan

HashMap mempermudah proses autentikasi karena pencarian data user dapat dilakukan secara cepat.

3.2.3 Fitur Logout

Fitur logout digunakan ketika pengguna ingin keluar dari sistem.

Proses

1. User memilih menu logout.
2. Sistem mengembalikan tampilan ke menu utama.

Output

- User keluar dari akun.

Struktur Data yang Digunakan

Tidak menggunakan struktur data khusus.

3.2.4 Fitur Tambah Aspirasi

Fitur tambah aspirasi digunakan citizen untuk membuat laporan baru terkait masalah publik.

Input

- Judul laporan
- Deskripsi laporan
- Kategori
- Lokasi

Proses

1. Citizen mengisi seluruh data laporan.
2. Sistem membuat objek Aspiration baru.
3. Status awal laporan diatur menjadi PENDING.
4. Sistem menyimpan laporan ke HashMap aspirasi.
5. Sistem memasukkan laporan ke LinkedList antrean verifikasi admin.
- 6. Sistem memperbarui verificationQueueMap**
- 7. Laporan diindeks ke invertedIndex untuk keperluan pencarian keyword.**
- 8. Data disimpan ke aspirations.txt dan verqueue.txt.**

Output

- Laporan berhasil dibuat dan masuk antrean verifikasi.

Struktur Data yang Digunakan

- `HashMap<String, Aspiration>`
- `LinkedList<Aspiration>`
- **`HashMap<String, LinkedList<String>>` (inverted index)**

Alasan Penggunaan

`HashMap` digunakan untuk menyimpan dan mencari laporan berdasarkan ID secara cepat, `Inverted index` digunakan untuk lookup secara cepat sedangkan `LinkedList` digunakan sebagai antrean FIFO laporan masuk.

3.2.4 Fitur Liat Aspirasi

Fitur lihat aspirasi digunakan citizen untuk melihat seluruh laporan publik beserta status dan jumlah upvote.

Proses

1. Sistem mengambil seluruh data aspirasi.
2. **Sistem memisahkan aspirasi milik sendiri dan aspirasi publik lain.**
3. Sistem menampilkan informasi laporan seperti judul, kategori, status, dan jumlah upvote.

Output

- **Daftar aspirasi saya dan daftar aspirasi publik lainnya ditampilkan.**

Struktur Data yang Digunakan

`HashMap<String, Aspiration>`

Alasan Penggunaan

`HashMap` mempermudah pengelolaan dan pengambilan data laporan secara efisien.

3.2.5 Fitur Cari Aspirasi (Pergantian Struktur Data)

Fitur diganti dari yang awalnya mencari berdasarkan ID laporan dan hashmap menjadi mencari laporan menggunakan inverted Index dan keyword yang lebih mempermudah pencarian.

Input

- **Kata kunci pencarian**

Proses

1. User memasukkan **keyword**.
2. Sistem mencari laporan dengan **inverted index**.
3. Jika ditemukan, sistem menampilkan detail laporan.

Output

- **Daftar aspirasi yang relevan dengan kata kunci.**
- **Pilihan untuk melihat detail aspirasi tertentu.**

Struktur Data yang Digunakan

Inverted index dengan implementasi `HashMap<String, LinkedList<String>>`

Alasan Penggunaan

Inverted Index memungkinkan pencarian laporan berdasarkan keyword dengan kompleksitas lookup $O(1)$ per kata kunci. Dibandingkan pencarian linear $O(n)$, pendekatan ini jauh lebih efisien saat jumlah laporan bertambah besar.

3.2.7 Fitur Upvote Aspirasi

Fitur upvote digunakan oleh citizen untuk mendukung laporan masyarakat lain agar menjadi prioritas.

Input

- ID laporan

Proses

1. Citizen memilih laporan yang ingin didukung.
2. Sistem mengecek apakah username sudah pernah melakukan upvote.
3. Jika belum, username dimasukkan ke LinkedList upvotedUsers.
4. Sistem menambahkan jumlah upvote laporan.
5. Data disimpan ulang ke file.

Output

- Jumlah upvote laporan bertambah.

Struktur Data yang Digunakan

LinkedList<String>

HashMap<String, Aspiration>

Alasan Penggunaan

LinkedList digunakan untuk menyimpan daftar user yang sudah melakukan upvote dan mencegah duplicate upvote.

3.2.8 Fitur Verifikasi Aspirasi (Penambahan data structure)

Fitur verifikasi aspirasi digunakan admin untuk memvalidasi laporan masyarakat.

Input

- ID laporan
- Status verifikasi (disetujui atau tidak)

Proses

1. Admin memilih laporan dari antrean verifikasi.
2. Admin memasukkan ID laporan yang akan diverifikasi
3. Sistem memperbarui status laporan menjadi APPROVED atau REJECTED.
4. Laporan dihapus dari verificationQueue dan verificationQueueMap.
5. Jika disetujui, laporan dapat diproses ke tahap berikutnya.
6. Data disimpan ulang

Output

- Status laporan berhasil diperbarui.

Struktur Data yang Digunakan

- LinkedList<Aspiration>
- HashMap<String, Aspiration>

Alasan Penggunaan

LinkedList digunakan karena laporan diverifikasi berdasarkan urutan laporan masuk menggunakan konsep FIFO sedangkan hash map digunakan untuk pencarian cepat.

3.2.9 Fitur Penentuan Prioritas

Fitur ini digunakan admin untuk menentukan tingkat prioritas laporan.

Input

- ID laporan
- Tingkat prioritas

Proses

1. Admin memilih laporan yang telah diverifikasi.
2. Admin memasukkan nilai S_auth dan S_safe.
3. Sistem menghitung preview totalScore
4. Admin mengkonfirmasi dan skor dikunci

5. Laporan dipindah dari approvedUnscoredList ke readyToDistributeList
6. Prioritas disimpan pada object Aspiration.

Output

- Prioritas laporan berhasil diperbarui.
- Laporan siap untuk didistribusikan ke institusi.

Struktur Data yang Digunakan

LinkedList<Aspiration>

PriorityQueue<Aspiration>

Alasan Penggunaan

LinkedList digunakan sebagai buffer bertahap antar fase alur kerja approved, scored, distributed. PriorityQueue digunakan agar laporan prioritas tinggi diproses lebih dahulu oleh institusi. Sistem SIGAP mengadopsi algoritma cerdas yang digunakan untuk menyaring data, namun keputusan akhir tetap berada di tangan otoritas manusia (Admin). Hal ini bertujuan untuk mencegah manipulasi data oleh pelapor dan memastikan bahwa prioritas keselamatan nyawa divalidasi secara profesional.

Komponen Perhitungan Skor

Skor akhir (1.0 - 10.0) disusun dengan struktur bobot yang memberikan dominasi pada aspek keselamatan dan keputusan administratif:

Komponen	Bobot	Deskripsi Sumber Data
Admin Authority (S_auth)	30%	Keputusan Final Admin: Validasi kebenaran dan urgensi laporan.

Safety & Health (S_safe)	25%	Tingkat ancaman terhadap nyawa/kesehatan (1-10).
Scale of Impact (S_scale)	15%	Akumulasi <i>Upvote</i> luas cakupan populasi yang terdampak (1-10).
Category Weight (S_cat)	15%	Bobot tetap sektor (Kesehatan = 10, Admin = 2).
System (S_sys)	15%	Aging (waktu tunggu).

Formula Matematis dengan Final Authority

Formula ini memastikan bahwa input algoritma (Safety, Scale, Category) hanya menyumbang 70% dari total skor, sementara 30% sisanya adalah kendali murni dari Admin:

$$\text{TotalScore} = (S_{\text{auth}} * 0.30) + (S_{\text{safe}} * 0.25) + (S_{\text{scale}} * 0.15) + (S_{\text{cat}} * 0.15) + (S_{\text{ext}} * 0.15)$$

Mekanisme Kerja & Kendali Admin

1. Verification Gate: Setiap laporan yang masuk dari Citizen berstatus pending. Laporan ini tidak akan masuk ke antrean Institusi sebelum Admin memberikan nilai S_{auth} .
2. Filtering Exaggeration: Jika seorang pelapor memasukkan nilai *Safety* 10 (Sangat Berbahaya) namun deskripsi menunjukkan hanya masalah minor, Admin dapat memberikan S_{auth} rendah (misal: 1 atau 2). Hal ini akan secara drastis menurunkan total skor laporan tersebut meskipun nilai variabel lainnya tinggi.
3. Emergency Override: Sebaliknya, jika terdapat kondisi darurat yang belum terdeteksi secara otomatis, Admin dapat memberikan nilai S_{auth} maksimal (10) untuk memaksa laporan tersebut naik ke puncak antrian eksekusi.

4. Data Integrity: Admin bertindak sebagai validator yang memastikan bahwa *Scale of Impact* yang diklaim (misal: "Satu Kota") benar-benar sesuai dengan realita lapangan sebelum skor dikunci.

Alur Antrean Ganda (Dual-Queue System)

Sistem mengimplementasikan dua tahap pemrosesan menggunakan struktur data yang dioptimalkan:

- **Antrean Prioritas (Institusi) *PriorityQueue (Max-Heap)*:**

Setelah Admin memberikan "Final Authority Score", laporan berpindah ke antrean ini. Di sini, laporan diurutkan berdasarkan Total Score. Laporan dengan nilai keselamatan tertinggi yang telah divalidasi Admin akan menempati posisi teratas untuk segera ditindaklanjuti.

Implementasi Otoritas Final pada Sistem

Setelah Admin memasukkan nilai, sistem akan melakukan *locking* (penguncian) pada skor tersebut. Institusi pelaksana tidak dapat mengubah urutan ini, sehingga akuntabilitas kerja instansi sepenuhnya bergantung pada daftar prioritas yang telah disahkan oleh Admin pusat.

3.2.10 Fitur Distribusi Institusi (Tambahan data structure)

Fitur distribusi institusi digunakan admin untuk mengirim laporan ke institusi yang sesuai dengan kategori laporan.

Input

- ID laporan
- institusi tujuan

Proses

1. Admin memilih laporan dari **readyToDistributeList**
2. Admin memilih institusi tujuan.
3. Sistem memasukkan laporan ke PriorityQueue milik institusi terkait.
4. Laporan siap diproses institusi.

Output

- Laporan berhasil dikirim ke institusi tujuan.
- Laporan masuk antrean PriorityQueue institusi sesuai skor.

Struktur Data yang Digunakan

PriorityQueue<Aspiration>

LinkedList<Aspiration>

Alasan Penggunaan

PriorityQueue membantu institusi memproses laporan berdasarkan tingkat prioritas tertinggi. LinkedList memastikan admin hanya melihat laporan yang benar benar siap di distribusi.

3.2.11 Fitur Proses Laporan Institusi (Tambahan Data Structure)

Fitur ini digunakan oleh institusi untuk menangani laporan yang diterima dari admin.

Input

- ID laporan

Proses

1. InstitutionAdmin melihat antrean laporan institusinya yang diurutkan berdasarkan skor.
2. InstitutionAdmin memilih laporan yang akan diproses.
3. Sistem menghapus laporan dari PriorityQueue institusi.

4. Status laporan diubah menjadi ON_PROGRESS.

Output

- Status laporan berhasil diperbarui menjadi sedang diproses.

Struktur Data yang Digunakan

PriorityQueue<Aspiration>

HashMap<String, LinkedList<Aspiration>>

Alasan Penggunaan

PriorityQueue memastikan laporan dengan prioritas tertinggi diproses terlebih dahulu.

Sebuah laporan tidak boleh dianggap selesai hanya karena instansi mengubah statusnya. Kriteria "Done" yang kredibel meliputi:

1. **Bukti Visual Terverifikasi: Instansi wajib mengunggah foto/video hasil perbaikan menggunakan link.**
2. **Laporan Penutupan (Closing Statement): Adanya penjelasan singkat mengenai apa yang telah dilakukan untuk menyelesaikan masalah tersebut.**

3.2.12 Fitur Registrasi Admin Institusi

Fitur ini digunakan oleh institusi untuk membuat akun admin khusus institusi.

Input

- Username
- Password
- Nama admin

Proses

1. Institution memasukkan data admin baru.
2. Sistem membuat object InstitutionAdmin.
3. Sistem menyimpan akun ke HashMap user.
4. Data disimpan di users.txt

Output

- Akun InstitutionAdmin berhasil dibuat.

Struktur Data yang Digunakan

HashMap<String, User>

Alasan Penggunaan

HashMap mempermudah penyimpanan dan pencarian akun admin institusi.

3.2.13 Fitur Dashboard Statistik

Fitur dashboard statistik digunakan admin untuk melihat ringkasan data laporan dalam sistem.

Menampilkan

- Total laporan
- Laporan selesai
- Laporan sedang diproses
- Laporan pending
- Jumlah laporan per institusi
- Laporan dengan upvote tertinggi

- Trending issue

Proses

1. Sistem membaca semua nilai dari variable global (statTotal, statDone, statOnProgress, statPending, statPerInstitusi, statPerKategori, statTopUpvote).
2. Sistem menampilkan hasil statistik pada dashboard admin.

Output

- Informasi statistik laporan pada sistem SIGAP.

Struktur Data yang Digunakan

HashMap<String, Integer>

Alasan Penggunaan

HashMap mempermudah pengelolaan dan pengambilan data laporan untuk kebutuhan statistik sistem.

3.2.14 Fitur Komentar Aspirasi (Fitur Tambahan)

Fitur komentar memungkinkan citizen untuk memberikan komentar publik pada laporan aspirasi mana pun, meningkatkan transparansi dan diskusi komunitas.

Input:

- ID aspirasi yang ingin dikomentari
- Teks komentar

Proses:

- Citizen memilih aspirasi dari daftar yang ditampilkan.
- Sistem menampilkan detail aspirasi beserta komentar yang sudah ada.
- Citizen memasukkan teks komentar.

- Sistem membuat object `CommentAspiration` dengan username, isi komentar, dan timestamp otomatis.
- Komentar ditambahkan ke `LinkedList<CommentAspiration>` milik aspirasi tersebut.
- Data komentar disimpan ke `comments.txt`.

Struktur Data yang Digunakan

`LinkedList<CommentAspiration>`

3.2.15 Fitur Lihat Antrian Dual Queue (Fitur Tambahan)

Fitur yang dapat menampilkan laporan antrian setiap institusi yang ada di dalam sistem

Proses:

- Sistem menampilkan semua institusi beserta laporan dalam antrian `PriorityQueue`.
- Sistem menampilkan laporan yang sedang diproses (`ON_PROGRESS`) per institusi.
- Informasi: ID, judul, skor, dan label prioritas ditampilkan per laporan.

Output:

Tampilan lengkap status antrian laporan per institusi.

Struktur Data yang Digunakan

`PriorityQueue<Aspiration>`.

`HashMap<String, LinkedList<Aspiration>>`.

Alasan Penggunaan

PriorityQueue memungkinkan tampilan laporan terurut berdasarkan prioritas. **institutionOnProgressMap** memungkinkan mencari laporan **ON_PROGRESS** per institusi dengan cepat.

3.2.16 Fitur Update Status Laporan (Fitur Tambahan)

Fitur ini memungkinkan **InstitutionAdmin** untuk menandai laporan yang sedang “On progress” menjadi “DONE” disertai closing statement dan bukti penyelesaian.

Input

- ID laporan yang sedang diproses
- Closing statement (penjelasan tindakan yang dilakukan)
- Bukti penyelesaian dalam link URL foto

Proses:

- **InstitutionAdmin** memilih laporan.
- **InstitutionAdmin** memasukkan closing statement dan bukti penyelesaian.
- Status laporan diubah menjadi **DONE**.

Output:

Laporan ditandai selesai dengan closing statement tercatat.

Struktur Data yang Digunakan

HashMap<String, LinkedList<Aspiration>>

3.2.17 Fitur Riwayat Laporan Selesai (Fitur Tambahan)

Fitur pada **institution admin** yang dapat menunjukan riwayat laporan yang telah diselesaikan sebelumnya

Proses

1. **InstitutionAdmin** memilih menu Riwayat Laporan Selesai.
2. Sistem mengambil data dari **institutionDoneMap** dengan **O(1)**.

3. Daftar laporan selesai ditampilkan: ID, judul, prioritas, dan closing statement.
4. InstitutionAdmin dapat memilih ID untuk melihat detail lengkap laporan.

Output

- Daftar laporan yang telah selesai ditangani institusi.

Struktur Data yang Digunakan

HashMap<String,LinkedList<Aspiration>>

Alasan Penggunaan

HashMap memungkinkan akses riwayat per institusi secara $O(1)$ tanpa iterasi seluruh aspirationMap. Ini sangat efisien saat jumlah laporan bertambah besar seiring berjalannya waktu.

3.2.18 Fitur Kelola Institusi (Fitur Tambahan)

Admin dapat menambah dan menghapus institusi dari sistem melalui menu Kelola Institusi.

Tambah Institusi:

- Admin memasukkan nama institusi baru.
- Sistem membuat object Institution baru dan menyimpannya ke institutionMap.
- Data disimpan ke institutions.txt.

Sub-fitur Tambah Institusi:

- Admin memilih institusi yang akan dihapus.
- Sistem memvalidasi: hapus diblokir jika institusi masih memiliki laporan dalam antrean atau laporan ON_PROGRESS.
- Jika aman, institusi dihapus dari institutionMap, institutionOnProgressMap, institutionDoneMap, dan statPerInstitusi.
- Data disimpan ulang.

Struktur data yang digunakan

HashMap<String,Institution>

Alasan penggunaan

HashMap memungkinkan penambahan dan penghapusan institusi secara $O(1)$.

3.2.19 Fitur penyimpanan data (Fitur Tambahan)

Fitur penyimpanan data menggunakan file txt digunakan menyimpan data user dan aspirasi yang telah dimasukan sehingga dapat diakses kembali setelah sistem telah berhenti.

Proses

- 1. Sistem membuat file bentuk .txt jika belum ada**
- 2. Sistem save data setiap membuat aspirasi atau user baru**
- 3. Setelah ditutup dan dinyalakan kembali sistem melakukan proses memuat data yang ada di file.txt yang sudah dibuat**

Output

- File .txt yang berisi kumpulan data user,aspirasi,institusi,counter yang telah diinputkan**

Alasan Penggunaan

Sistem dapat menggunakan kembali data yang telah di masukan di penggunaan sebelumnya sehingga tidak perlu melakukan input ulang data.

BAB IV TIMELINE

Pada bab ini kami cantumkan timeline proses pengembangan aplikasi SIGAP sesuai dengan jadwal yang telah disepakati bersama oleh seluruh anggota tim. Setiap tahapan pengerjaan disusun secara sistematis agar proses pembangunan sistem dapat berjalan terarah, terukur, dan selesai tepat waktu. Detail timeline dapat dilihat pada Tabel 5.1.

Tabel 5.1. Timeline Pengembangan Program

Hari, Tanggal	Yang Dikerjakan	Person In Charge
Senin, 11 Mei 2026	Menyampaikan Fitur yang ada dalam aplikasi	ARYA CAVIN DARREN
Selasa, 12 Mei 2026	Menyelesaikan Proposal	ARYA CAVIN DARREN
Senin, 18 Mei 2026	1. Fitur Register 2. Fitur Login 3. Fitur Logout 4. Fitur Tambah Aspirasi	ARYA CAVIN DARREN DARREN
Senin, 25 Mei 2026	5. Fitur Lihat Aspirasi 6. Fitur Upvote Aspirasi 7. Fitur Cari Aspirasi 8. Fitur Verifikasi Aspirasi 9. Fitur Penentuan Prioritas	ARYA ARYA CAVIN CAVIN DARREN
Selasa, 2 June 2026	10. Fitur Distribusi Institusi 11. Fitur Proses Laporan Institusi 12. Fitur Registrasi Admin Institusi 13. Fitur Statistik Dashboard 14. Komentar Aspirasi 15. Lihat Antrian Dual Queue 16. Update Status Laporan	ARYA ARYA CAVIN DARREN ARYA ARYA CAVIN

	17. Riwayat Laporan Selesai 18. Kelola Institusi 19. Penyimpanan Data	DARREN ARYA CAVIN
--	---	-------------------------

DAFTAR PUSTAKA

- [1] United Nations, "Goal 16: Peace, Justice and Strong Institutions," in *Transforming Our World: The 2030 Agenda for Sustainable Development*, New York: United Nations, 2015.
- [2] OECD Observatory of Public Sector Innovation, "SP4N-LAPOR! Indonesia's National Complaint Handling Management System," OECD OPSI, 2021. [Online]. Available: <https://oecd-opsi.org/innovations/sp4n-lapor-indonesias-national-complaint-handling-management-system/>
- [3] KemenPANRB, "Laporan Kinerja SP4N-LAPOR!," Kementerian Pendayagunaan Aparatur Negara dan Reformasi Birokrasi, Jakarta, Indonesia, 2022.
- [4] R. Pratama, A. Manasikana, and N. A. Fadzlina, "Improving the Quality of Public Services Through the SP4N-LAPOR! Complaint Application at the Ministry of State Apparatus Empowerment and Bureaucratic Reform," *Jurnal Ilmiah Wahana Bhakti Praja*, vol. 13, no. 2, 2023. [Online]. Available: <https://ejournal.ipdn.ac.id/JIWP/article/view/3627>
- [5] M. G. Mali and A. Prastyawan, "Pemanfaatan Data Pengaduan SP4N LAPOR Dalam Perumusan Kebijakan Publik Di Pemerintah Kabupaten Madiun," *Ganaya: Jurnal Ilmu Sosial dan Humaniora*, 2026. [Online]. Available: <https://jayapanguspress.penerbit.org/index.php/ganaya/article/view/5159>
- [6] V. K. Srinivas and K. V. Teja, "Citizen Complaint Management System," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, 2026. [Online]. Available: <https://www.ijarsct.co.in/Paper32687.pdf>
- [7] Tempo, "Indeks Kepercayaan Publik terhadap Lembaga Negara 2026," Tempo Media Group, Jakarta, Indonesia, 2026.
- [8] M. Hutahaean, I. J. Eunike, and A. D. K. Silalahi, "Do Social Media, Good Governance, and Public Trust Increase Citizens' E-Government Participation? Dual Approach of PLS-SEM and fsQCA," *Human Behavior and Emerging Technologies*, vol. 2023, pp. 1–19, 2023, doi: 10.1155/2023/9988602.
- [9] United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*, New York: United Nations, 2015.
- [10] A. Setiawan et al., "Impact of Government Digital Transformation on Citizen Trust and Participation: Evidence from Gowa Regency, Indonesia," *Frontiers in Human Dynamics*, 2025, doi: 10.3389/fhumd.2025.1700582.
- [11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022. ISBN: 978-0-262-04630-5. [Online]. Available: https://cdn.manesht.ir/19908___Introduction%20to%20Algorithms.pdf
- [12] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley Professional, 2011. ISBN: 978-0-321-57351-3. [Online]. Available: <https://algs4.cs.princeton.edu/home/>

- [13] M. A. Weiss, Data Structures and Algorithm Analysis in Java, 3rd ed. Upper Saddle River, NJ: Pearson, 2012. ISBN: 978-0-132-57627-7. [Online]. Available: <https://ebooks.karbust.me/Technology/Data.Structures.and.Algorithm.Analysis.in.Java.3rd.Edition.Nov.2011.pdf>
- [14] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python, 1st ed. Hoboken, NJ: Wiley, 2013. ISBN: 978-1-118-29027-9. [Online]. Available: <https://nibmehub.com/opac-service/pdf/read/Data%20Structures%20and%20Algorithms%20in%20Python.pdf>
- [15] S. S. Skiena, The Algorithm Design Manual, 3rd ed. Cham, Switzerland: Springer, 2020. ISBN: 978-3-030-54255-9. [Online]. Available: <https://sureshcseit.wordpress.com/wp-content/uploads/2021/04/skienathealgorithmdesignmanual.pdf>
- [16] A. Castro, M. Areias, and R. Rocha, “Performance Evaluation of Separate Chaining for Concurrent Hash Maps,” *Mathematics*, vol. 13, no. 17, p. 2820, 2025. doi: 10.3390/math13172820.